Assignment 2

Team Members: Robin Sanssi (rosa24) Mirnes Mrso (mimr24)

Group Number: 35

Repository Link: https://github.com/mirNNes/bsv-edutask

Work distribution

The work was divided between us both. Mirnes prepared the initial drafts for parts of the assignment, and Robin reviewed and improved different parts. All parts were discussed and checked by us both.

1. Mocking

Explanation of mocking

Mocking is a testing technique in which real dependencies of a system are replaced with controlled, simulated objects. Instead of interacting with actual components such as databases or external services, a mock object is used to imitate their behavior. The tester can define in advance what the mock should return or how it should behave under certain conditions.

Purpose of mocking in unit testing

The main purpose of mocking in unit testing is to isolate the unit under test from its dependencies. By replacing external components with mocks, the test focuses only on the behavior of the specific method being tested. This ensures that the test results are not influenced by unrelated parts of the system.

Additionally, mocking allows the tester to simulate different scenarios, including edge cases and error conditions, which might be difficult or impossible to reproduce using real components. It also improves test reliability and repeatability, since the behavior of the mock is fully controlled and deterministic.

2. Unit Testing

Test Case Design

The test cases for the get_user_by_email method were derived based on the method’s specification provided in the docstring. The docstring was used as the test oracle, defining the expected behavior of the method for different inputs.

We used the 4-step test design technique to create the test cases in a structured way.

Step 1: Identify action and expected outcomes

The action is:

- Call get_user_by_email(email).

The possible expected outcomes are:

- The matching user object is returned.
- The first matching user object is returned and a warning message is printed.
- None is returned.
- ValueError is raised.
- A database exception is propagated.

Step 2: Identify conditions

The outcome depends on whether the email input is valid and on what the DAO returns when it searches for the email.

Condition	Possible values
Email validity	valid email / invalid email
DAO lookup result	one matching user / multiple matching users / no matching users / database failure

The docstring says that a valid email should follow the format <local-part>@<domain>.<host>. Because of this, jane@doe.com is used as a valid email, while both invalid-email and jane@doe are invalid.

Step 3: Determine combinations

The relevant combinations are listed below. When the email is invalid, the DAO result does not matter, because the method should raise ValueError before calling the DAO.

#	Email validity	DAO lookup result	Relevant?
1	valid email	one matching user	yes
2	valid email	multiple matching users	yes
3	valid email	no matching users	yes
4	valid email	database failure	yes
5	invalid email	any DAO result	collapsed, because the DAO should not be called

Step 4: Define expected outcomes

From these combinations, we derived the following test cases:

Test case	Email input	Mocked DAO behavior	Expected outcome
TC1	jane@doe.com	returns [user]	the user object is returned
TC2	jane@doe.com	returns [user1, user2]	the first user object is returned and a warning message is printed
TC3	jane@doe.com	returns []	None is returned

TC4	invalid-email	DAO is not called	ValueError is raised
TC5	jane@doe	DAO is not called	ValueError is raised
TC6	jane@doe.com	raises Exception	the exception is propagated

Test Implementation

The test cases were implemented using the Pytest framework. The structure of the tests follows the Arrange-Act-Assert (AAA) pattern, where inputs are prepared, the method is executed, and the result is verified.

Mocking was applied using unittest.mock.MagicMock to simulate the DAO dependency. This allowed full control over the return values of the find() method without relying on a real database. This is important because the database is outside the scope of this unit test.

Fixtures were used to create reusable test setup for the mocked DAO and the UserController instance. Additionally, the capsys fixture was used to capture printed output when testing the behavior for multiple matching users.

The invalid email test cases also check that the DAO is not called. This is done because invalid input should be rejected before the method tries to search in the database.

The implementation of the tests can be found in the file:

https://github.com/mirNNes/bsv-edutask/blob/master/backend/test/unit/test_usercontroller.py

Test Execution Result

The tests were executed with the following command:

```
.venv/bin/python -m pytest -m unit
```

Test execution output part 1

```
============================================= test session starts ==============================================
platform darwin -- Python 3.12.5, pytest-8.3.5, pluggy-1.6.0
rootdir: /Users/robin/dbwebb-kurser/sysver/bsv-edutask/backend
configfile: pytest.ini
testpaths: test
plugins: cov-6.1.0
collected 31 items / 25 deselected / 6 selected

test/unit/test_usercontroller.py ..F.F.                                                                  [100%]

=================================================== FAILURES ===================================================
_______________________________ test_get_user_by_email_returns_none_when_no_match _______________________________

controller = <src.controllers.usercontroller.UserController object at 0x102e749e0>
mocked_dao = <MagicMock id='4343679888'>

    @pytest.mark.unit
    def test_get_user_by_email_returns_none_when_no_match(controller, mocked_dao):
        mocked_dao.find.return_value = []

>       result = controller.get_user_by_email('jane@doe.com')

test/unit/test_usercontroller.py:45:
- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -

self = <src.controllers.usercontroller.UserController object at 0x102e749e0>, email = 'jane@doe.com'

    def get_user_by_email(self, email: str):
        """Given a valid email address of an existing account, return the user object contained in the database asso
ciated
        to that user. For now, do not assume that the email attribute is unique. Additionally print a warning messag
e containing the email
        address if the search returns multiple users.

        parameters:
            email -- an email address string

        returns:
            user -- the user object associated to that email address (if multiple users are associated to that email
: return the first one)
            None -- if no user is associated to that email address

        raises:
            ValueError -- in case the email parameter is not valid (i.e., conforming <local-part>@<domain>.<host>)
            Exception -- in case any database operation fails
        """

        if not re.fullmatch(emailValidator, email):
            raise ValueError('Error: invalid email address')

        try:
            users = self.dao.find({'email': email})
            if len(users) == 1:
                return users[0]
            else:
                print(f'Error: more than one user found with mail {email}')
>               return users[0]
E               IndexError: list index out of range

src/controllers/usercontroller.py:37: IndexError
```

Test execution output part 2

```
---------------------------------------------- Captured stdout call ----------------------------------------------
Error: more than one user found with mail jane@doe.com
____________________ test_get_user_by_email_raises_value_error_for_email_without_domain_host _____________________

controller = <src.controllers.usercontroller.UserController object at 0x102e96ff0>
mocked_dao = <MagicMock id='4343820224'>

    @pytest.mark.unit
    def test_get_user_by_email_raises_value_error_for_email_without_domain_host(controller, mocked_dao):
>       with pytest.raises(ValueError):
E       Failed: DID NOT RAISE <class 'ValueError'>

test/unit/test_usercontroller.py:60: Failed
---------------------------------------------- Captured stdout call ----------------------------------------------
Error: more than one user found with mail jane@doe
================================================ tests coverage ================================================
_______________________________ coverage: platform darwin, python 3.12.5-final-0 _______________________________

Name                                  Stmts   Miss  Cover   Missing
-------------------------------------------------------------------
src/controllers/__init__.py               0      0   100%
src/controllers/controller.py            31     22    29%   24-27, 44-47, 59-62, 80-84, 99-103
src/controllers/taskcontroller.py        68     68     0%   1-139
src/controllers/todocontroller.py        21     21     0%   1-40
src/controllers/usercontroller.py        24      5    79%   42-46
src/util/dao.py                          67     51    24%   25-40, 54-65, 79-83, 101-118, 134-141, 156-162, 170-173, 1
84
src/util/daos.py                          6      3    50%   14-16
src/util/helpers.py                      30     19    37%   16, 20, 33-39, 45, 58-64, 73-77
src/util/validators.py                    7      4    43%   13-16
-------------------------------------------------------------------
TOTAL                                   254    193    24%
========================================== short test summary info ==========================================
FAILED test/unit/test_usercontroller.py::test_get_user_by_email_returns_none_when_no_match - IndexError: list index
out of range
FAILED test/unit/test_usercontroller.py::test_get_user_by_email_raises_value_error_for_email_without_domain_host - F
ailed: DID NOT RAISE <class 'ValueError'>
==================================== 2 failed, 4 passed, 25 deselected in 0.17s ====================================
```

The test execution showed that four out of six test cases passed successfully. Two test cases failed.

The first failed test case was the scenario where no user is found. According to the oracle, the method should return None. Instead, the method tried to return the first item from an empty list, which caused an IndexError.

The second failed test case was the scenario with the email jane@doe. According to the oracle, this email is invalid because it does not follow the format <local-part>@<domain>.<host>. The method did not raise ValueError for this input.

Coverage Interpretation

The coverage report shows that approximately 79% of the usercontroller.py file is covered by the implemented tests. This indicates that the main execution paths of the get_user_by_email method are exercised, including normal operation, invalid input handling, and exception scenarios.

The overall project coverage is lower, as only a specific part of the system was tested in this assignment. This is expected, since the focus was on a single method rather than the entire application.